

iTECify — Project Architecture & Presentation Blueprint

1. Project Overview

What is iTECify?

iTECify is a **real-time collaborative coding platform** — think Google Docs meets VS Code. Multiple developers can simultaneously write, edit, and execute code in the browser, with AI-powered assistance built in.

Problem It Solves

Traditional collaborative coding requires:

- Screen sharing (one person types, others watch)
- Git-based async workflows (slow feedback loops)
- Local environment setup for every participant

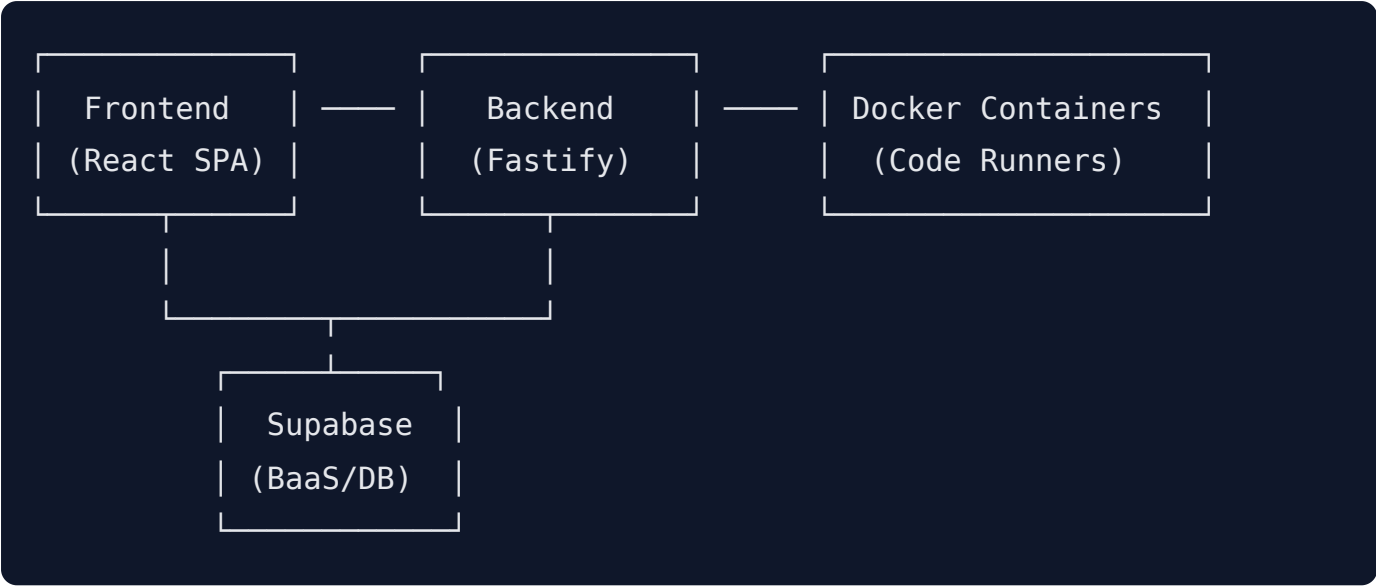
iTECify eliminates all three. Anyone with a browser can join a coding session, see other collaborators' cursors in real-time, edit the same files simultaneously, and **run the code instantly** — without installing any compiler, runtime, or dependency.

Who Interacts With It

Actor	Interaction
Developer (user)	Creates sessions, writes code, runs executions, uses AI chat
Collaborator	Joins via shared link, co-edits in real-time
AI Assistant	Provides code suggestions, explanations, debugging via Gemini
Docker Engine	Executes user code in isolated sandboxed containers
Supabase	Handles auth, database persistence, real-time sync

Main Architectural Idea

The system is split into three independent tiers:



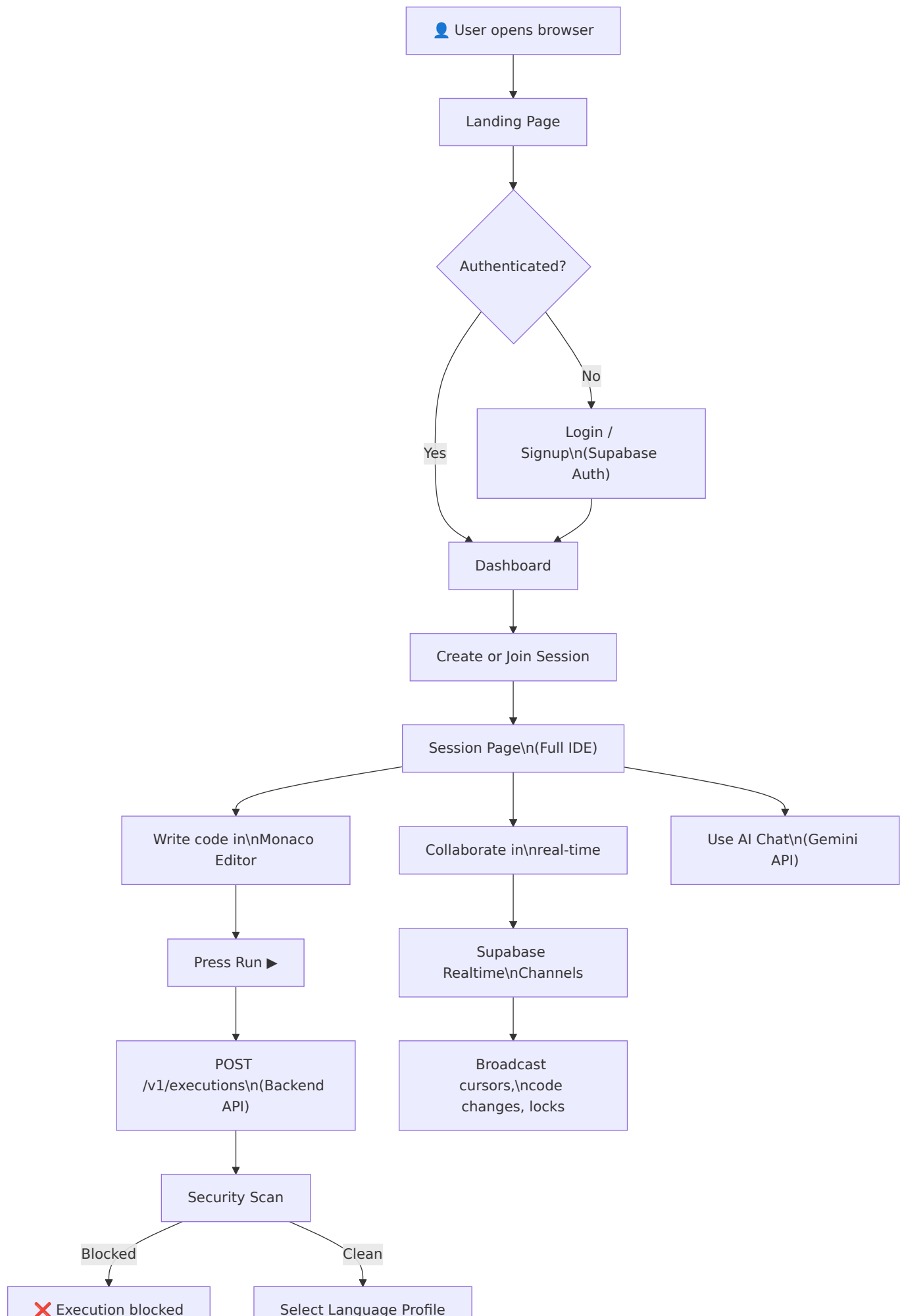
- 1. **Frontend:** React SPA with Monaco Editor, real-time collaboration, AI chat
- 2. **Backend:** Fastify API server that manages code execution pipelines
- 3. **Docker Runtime:** Ephemeral containers that run user code in sandboxed isolation
- 4. **Supabase:** Backend-as-a-Service providing authentication, PostgreSQL database, and real-time channels

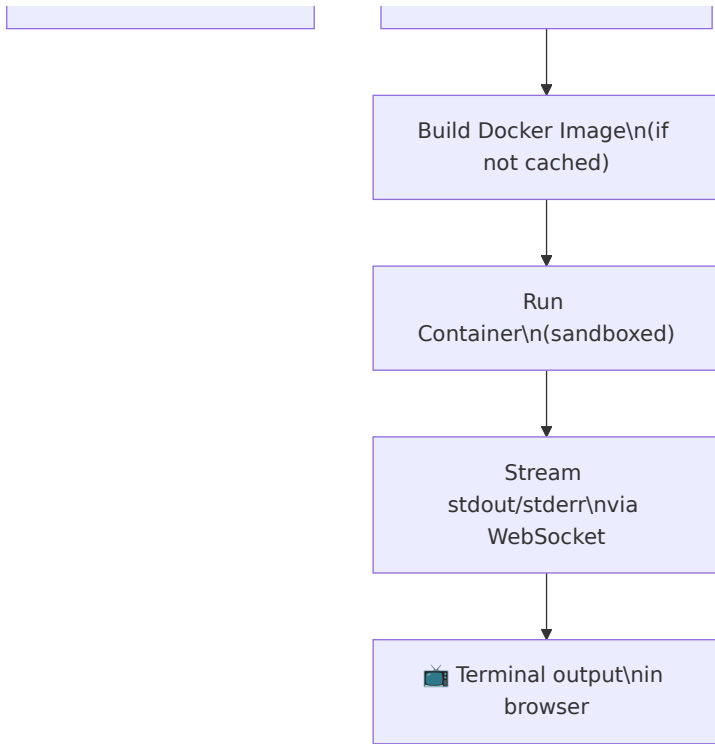
Most Important Components

Component	Purpose
Session Page	The full IDE experience (editor, file tree, terminal, AI, timeline)
Collaboration Store	Manages real-time cursor/code sync via Supabase channels
Execution Manager	Orchestrates the entire code-execution pipeline
Docker Sandbox	Builds and runs ephemeral containers per execution
Language Profiles	Template-driven runtime definitions (the "dynamic modules")
Security Scanner	Pre-execution code analysis to block dangerous patterns

2. Overall System Flow

End-to-End: From User Action to Result





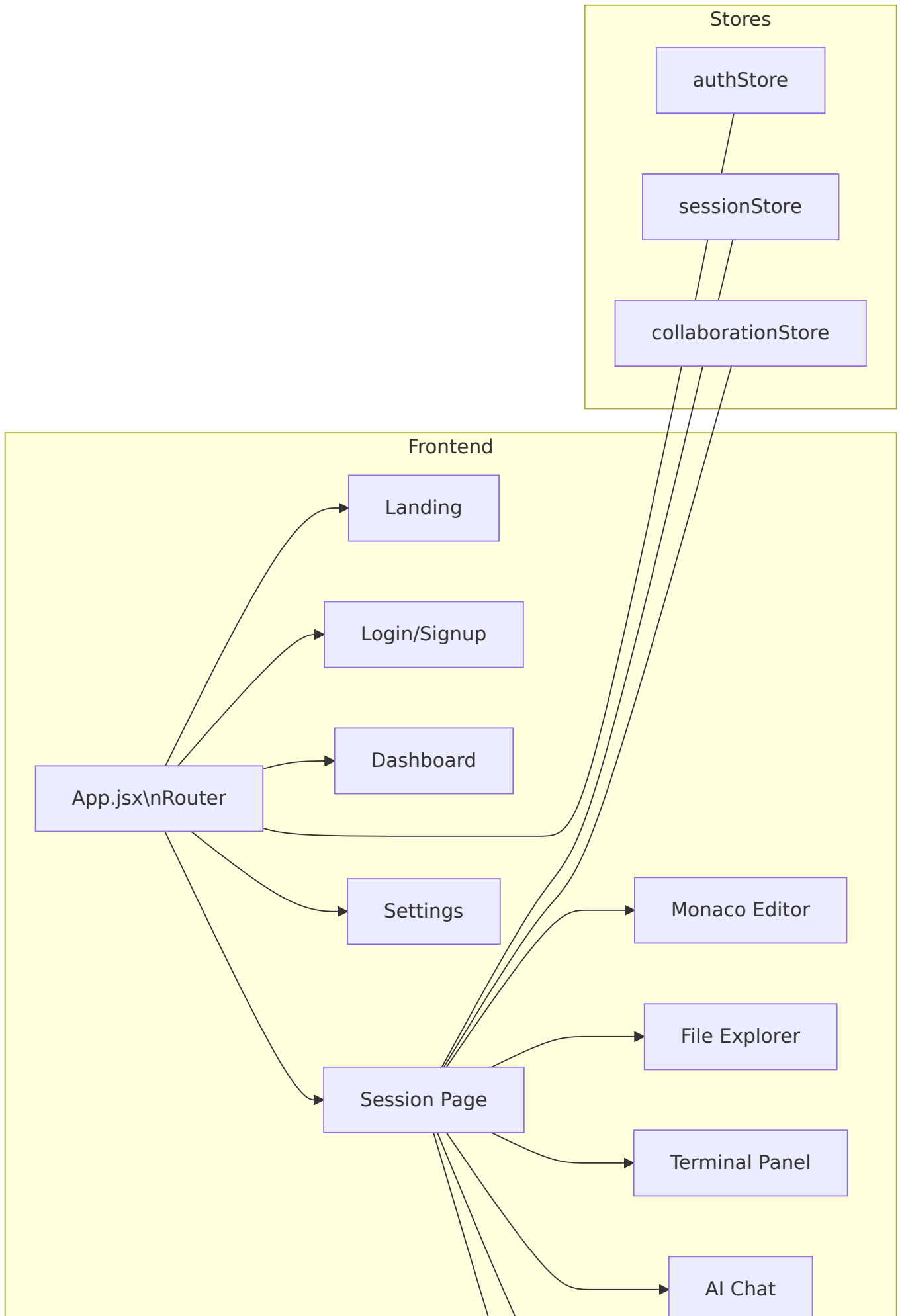
Detailed Flow Sequence

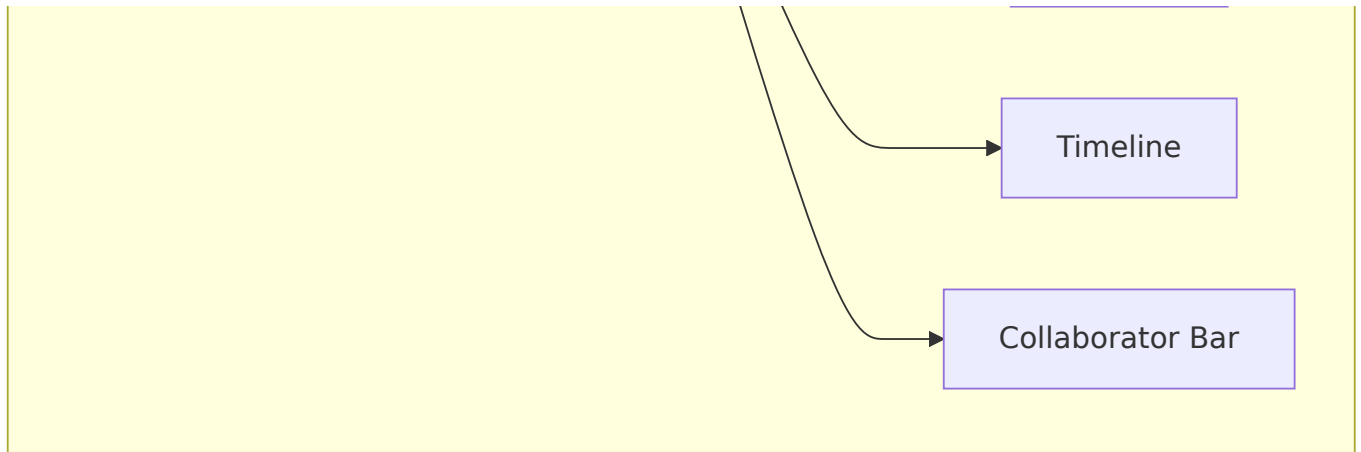
Step	What Happens	Component
1	User navigates to app	Browser → React Router
2	Auth state restored from Supabase session	authStore → Supabase
3	User creates/joins a session	Dashboard → sessionStore → Supabase DB
4	Session page loads with Monaco Editor	Session.jsx → Monaco Editor
5	Collaboration channel joined	collaborationStore → Supabase Realtime
6	User writes/edits code	Monaco → collaborationStore broadcasts changes
7	Other collaborators receive changes	Supabase channel → their Monaco instance
8	User clicks "Run"	TerminalPanel → <code>POST /v1/executions</code>
9	Backend scans code for security threats	securityScanner.js

10	Language profile selected (JS/Python/Rust)	languageProfiles.js
11	Dockerfile generated, context written to disk	dockerSandbox.js <code>prepareSandbox()</code>
12	Docker image built (or cache hit)	dockerSandbox.js <code>buildImageIfNeeded()</code>
13	Container launched with strict constraints	dockerSandbox.js <code>runSandboxedContainer()</code>
14	stdout/stderr streamed to frontend via WebSocket	executionRoutes.js WebSocket handler
15	Terminal displays output in real-time	TerminalPanel.jsx
16	Container exits → cleanup	executionManager.js → <code>removeSandboxContext()</code>
17	Timeline event recorded	sessionStore → <code>session_timeline_events</code> table

3. Architecture — Major Components

3.1 Frontend (React SPA)





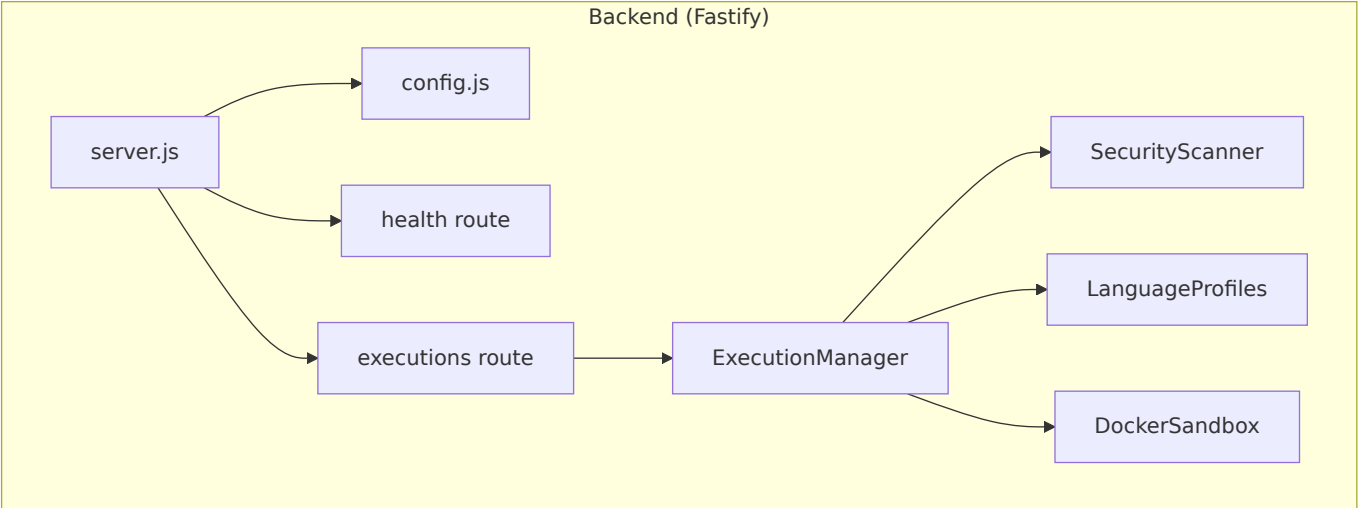
Component	Responsibility	Communicates With
Landing	Marketing page, feature showcase	Router
Login / Signup	Email + GitHub OAuth authentication	authStore → Supabase Auth
Dashboard	Session CRUD, listing user's sessions	sessionStore → Supabase DB
Session	Full IDE: editor, file tree, terminal, AI, timeline, collaboration	All stores, Backend API, Gemini API
Settings	AI configuration (custom API key, model)	authStore
ProtectedRoute	Auth guard for authenticated routes	authStore

Why it exists: The frontend is a single-page application to deliver the IDE-like experience entirely in the browser, with no server-side rendering needed.

3.2 State Management (Zustand Stores)

Store	Responsibility	Why It Exists
authStore	User session, auth actions, AI config persistence	Centralizes auth state; persists AI config to localStorage
sessionStore	Session CRUD, file system updates, timeline events	Decouples DB operations from UI components
collaborationStore	Real-time sync: cursors, line locks, code changes, terminal state	Manages Supabase Realtime channel lifecycle

3.3 Backend (Fastify API)



Component	Responsibility	Why It Exists
server.js	Fastify app bootstrap, middleware registration	Entry point
config.js	Environment-based configuration	Centralized config with safe defaults
executions.js	REST + WebSocket API for code execution	API boundary between frontend and execution engine
ExecutionManager	Orchestrates scan → build → run → stream pipeline	Core business logic; manages execution lifecycle
SecurityScanner	Pre-execution rule-based code analysis	Safety gate before running user code
LanguageProfiles	Runtime template definitions per language	Dynamic module system — defines how each language runs
DockerSandbox	Docker CLI orchestration (build, run, stop)	Isolation layer between backend logic and container runtime

3.4 Database (Supabase / PostgreSQL)

Table	Purpose
profiles	Display name uniqueness, auto-created on signup

<code>sessions</code>	Coding sessions with file system (stored as JSON in <code>code</code> column)
<code>session_timeline_events</code>	Time-travel debugging: every edit, create, delete event
<code>session_timeline_snapshots</code>	Periodic full file-system snapshots for timeline playback

Row-Level Security (RLS) is enabled on all tables, ensuring users can only access their own data (with collaborative access for shared sessions).

3.5 Real-Time Collaboration

Uses **Supabase Realtime Channels** (built on Phoenix Channels / WebSockets):

Channel	Events
<code>session: {sessionId}</code>	<code>cursor</code> , <code>line-lock</code> , <code>changes</code> , <code>full-sync</code> , <code>fs-change</code> , <code>terminal-state</code> , <code>terminal-lock-request</code> , <code>terminal-lock-grant</code> , <code>session-deleted</code>
Presence	<code>sync</code> , <code>join</code> , <code>leave</code> — tracks who's online

This enables **zero-latency collaborative editing** without a custom WebSocket server for the editor.

4. Technologies, Frameworks & Programming Languages

Frontend

Technology	Where Used	Why Chosen
JavaScript (ES Modules)	Entire frontend codebase	Industry standard for web
React 18	UI framework	Component model, rich ecosystem, Monaco integration
Vite	Build tool / dev server	Fast HMR, ESM-native, minimal config

React Router v6	Client-side routing	SPA navigation between pages
Zustand	State management	Lightweight alternative to Redux; no boilerplate
Monaco Editor	In-browser code editor	Same editor as VS Code; full language support
Supabase JS SDK	Auth, DB, Realtime	Unified client for all Supabase services
TailwindCSS 3	Styling	Utility-first CSS; rapid prototyping
Framer Motion	Animations	Declarative animations for React
GSAP	Advanced animations	Landing page effects, marquee scrolling
OGL	WebGL 3D graphics	Landing page 3D background effects
Lucide React	Icons	Consistent, tree-shakeable icon set
JSZip	File export	Download session files as ZIP
React Hot Toast	Notifications	Lightweight toast notifications
Google Fonts	Typography	Inter, Space Grotesk, Fira Code

Backend

Technology	Where Used	Why Chosen
Node.js 18	Runtime	Same language as frontend; npm ecosystem
Fastify 5	HTTP framework	High-performance, plugin architecture
@fastify/websocket	WebSocket support	Real-time execution streaming
@fastify/cors	CORS middleware	Cross-origin requests from frontend
Zod	Request validation	Type-safe schema validation
UUID	Execution ID generation	Unique identifiers for executions

dotenv	Environment config	Standard <code>.env</code> file loading
--------	--------------------	---

Infrastructure & Services

Technology	Where Used	Why Chosen
Docker	Code execution sandboxing	Isolation, resource limits, multi-language support
Supabase	Auth, PostgreSQL, Realtime	Managed BaaS — eliminates custom backend for auth/DB/realtime
Nginx	Production static file server	Serves built frontend assets in Docker container
Vercel	Alternative deployment	Serverless frontend hosting
Google Gemini API	AI assistant	Free-tier AI for code assistance

Programming Languages Used

Language	Where
JavaScript / JSX	Frontend + Backend (100% of application code)
SQL	Database schema and RLS policies
Dockerfile syntax	Container image definitions (frontend + backend + language runtimes)

5. Dynamic Modules — Technical Deep Dive

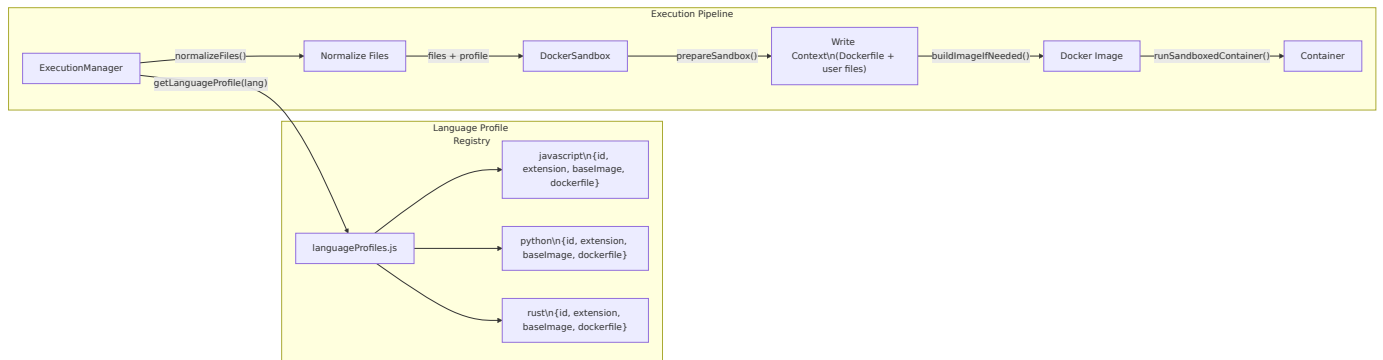
5.1 What "Dynamic Module" Means in This Project

In iTECify, the term "dynamic module" refers to the **Language Profile** system — a template-driven architecture that allows the backend to **dynamically construct and execute isolated runtime environments** for different programming languages, without any of those languages being pre-installed on the backend server.

[!IMPORTANT]

The backend itself is a pure Node.js application. It does not include Python, Rust, or any other runtime. Instead, it **generates Docker images on the fly** from language profile templates, building a complete execution environment per request.

5.2 Architecture



5.3 What Makes Modules Dynamic

Each language profile is a **self-contained specification** that defines everything needed to execute code in that language:

```
// From languageProfiles.js
{
  javascript: {
    id: "javascript",
    extension: "js", // File extension for the entry
point
    baseImage: "node:20-alpine", // Docker base image
    dockerfile: [ // Complete Dockerfile template
      "FROM node:20-alpine",
      "WORKDIR /workspace",
      "RUN addgroup -S runner && adduser -S runner -G runner",
      "COPY . .",
      "USER runner",
      'CMD ["node", "main.js"]',
    ].join("\n"),
  },
  // ... python, rust similarly defined
}
```

The dynamism comes from three properties:

Property	What It Does	Why It's Dynamic
extension	Determines the entry-point filename (<code>main.js</code> , <code>main.py</code> , <code>main.rs</code>)	The system adapts file creation based on the selected language
baseImage	Specifies which Docker base image to use	Different languages need fundamentally different runtimes
dockerfile	Complete Dockerfile template as a string	The entire container image is generated at runtime from this template

5.4 How Modules Are Discovered

```
// Simple registry lookup
export function getLanguageProfile(language) {
  return languageProfiles[language] || null;
}
```

Discovery is **key-based lookup** in a plain JavaScript object. The system validates the requested language against the registry at execution time.

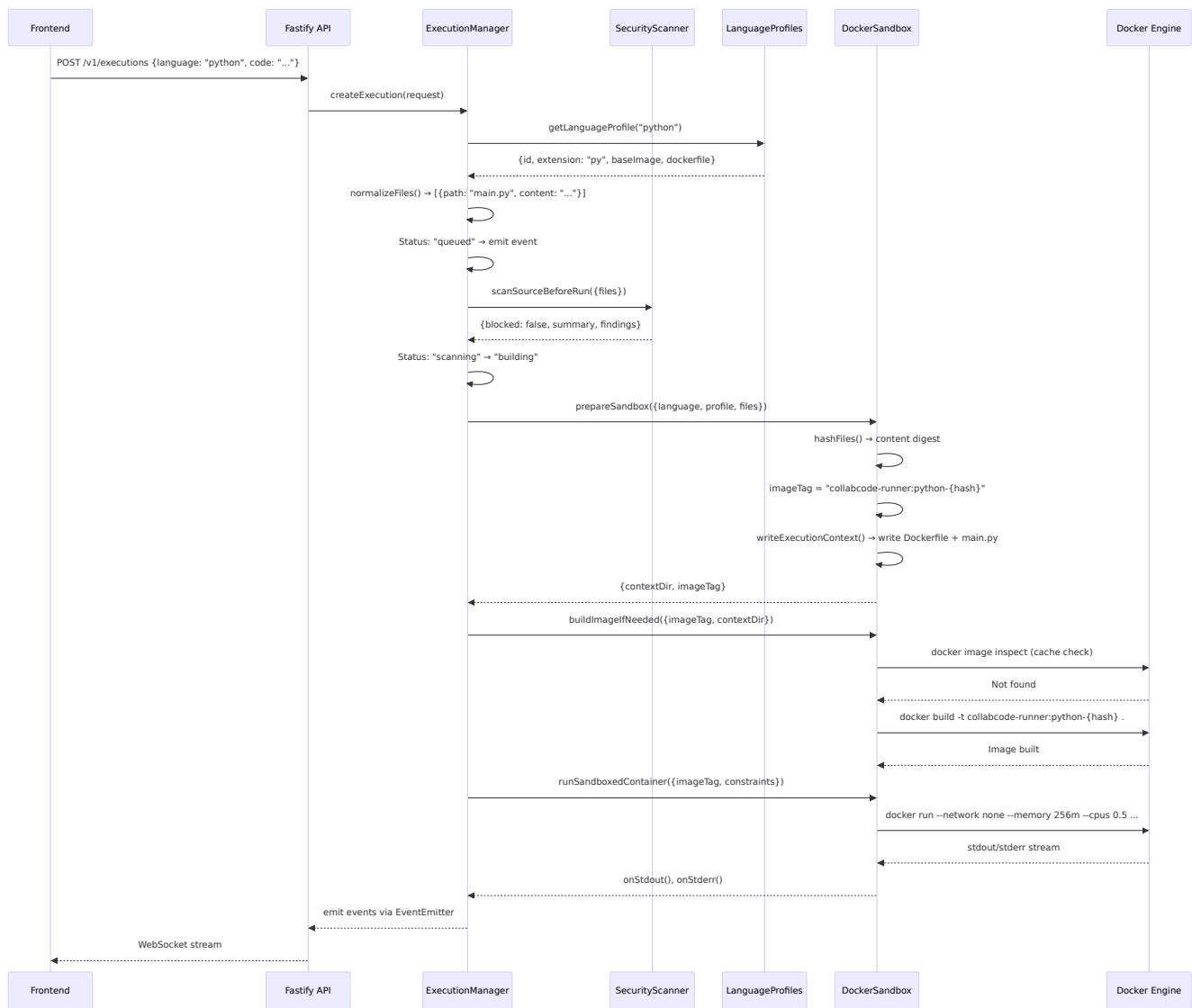
- **When:** At execution creation time (`createExecution()`)
- **How:** The API validates `language` against `z.enum(["javascript", "python", "rust"])` at the route level, then the `ExecutionManager` calls `getLanguageProfile(request.language)`
- **Failure:** Returns `null` → throws `"Unsupported language: ..."` error

5.5 How Modules Are "Loaded" and Initialized

The profiles are **statically defined** in code but **dynamically consumed** at runtime. This is a critical distinction:

Aspect	Behavior
Definition time	Build time (profiles are plain JS objects in source)
Selection time	Runtime (per execution request)
Image construction time	Runtime (Dockerfile is written to disk per execution)
Image build time	Runtime (Docker builds the image on first use)
Image caching	Content-addressed (same code + language = same image tag = cache hit)

5.6 The Execution Lifecycle of a Module



5.7 Content-Addressed Image Caching

A key optimization is the content-hash-based image tagging:

```
function hashFiles(files, language) {
  const hash = crypto.createHash("sha256");
  hash.update(language);
  for (const file of files.sort((a, b) => a.path.localeCompare(b.path)))
  {
    hash.update(file.path);
    hash.update(file.content);
  }
  return hash.digest("hex").slice(0, 20);
}

// Result: imageTag = `collabcode-runner:python-a1b2c3d4e5f6g7h8i9j0`
```

This means:

- **Same code** submitted twice → same hash → **Docker cache hit** → no rebuild
- **Different code** → different hash → new image built
- The image tag encodes both the **language** and the **content**, ensuring correctness

5.8 Module Interface / Contract

Every language profile must conform to this implicit contract:

```
interface LanguageProfile {
  id: string;           // Unique identifier (must match languageProfiles
                        // key)
  extension: string;    // File extension for entry point
  baseImage: string;    // Docker Hub base image reference
  dockerfile: string;   // Complete Dockerfile content as a string
}
```

The Dockerfile must:

1. Start from `baseImage`
2. Set up a workspace directory
3. Create a non-root `runner` user (security requirement)
4. Copy the user's files

5. Switch to the `runner` user
6. Define a `CMD` that runs the entry point

5.9 How to Add a New Language

Adding a new language (e.g., Go) requires exactly **two changes**:

1. Add profile to [languageProfiles.js](#):

```
go: {
  id: "go",
  extension: "go",
  baseImage: "golang:1.22-alpine",
  dockerfile: [
    "FROM golang:1.22-alpine",
    "WORKDIR /workspace",
    "RUN addgroup -S runner && adduser -S runner -G runner",
    "COPY . .",
    "RUN go build -o app main.go",
    "USER runner",
    'CMD ["/workspace/app"]',
  ].join("\n"),
},
```

2. Update validation in [executions.js](#):

```
language: z.enum(["javascript", "python", "rust", "go"]),
```

No other backend changes needed. The execution pipeline, Docker sandbox, security scanner, and WebSocket streaming all work unchanged.

5.10 Module Isolation

Modules are isolated at **multiple levels**:

Isolation Layer	Mechanism
Network	<code>--network none</code> — no internet access from container

Filesystem	<code>--read-only</code> root filesystem + restricted <code>/tmp</code>
Privileges	<code>--cap-drop ALL</code> , <code>--security-opt no-new-privileges</code> , <code>non-root</code> <code>runner user</code>
Resources	<code>--memory 256m</code> , <code>--cpus 0.5</code> , <code>--pids-limit 128</code>
Time	Configurable timeout (default 10 seconds)
Output	Max output bytes limit (default 200KB)

5.11 Error Handling

Error Scenario	How It's Handled
Docker not available	Detected at startup; executions fail gracefully with clear error message
Unsupported language	<code>getLanguageProfile()</code> returns <code>null</code> → throws before any Docker work
Security scan blocks	Execution set to <code>"blocked"</code> status; no Docker operations performed
Docker build fails	Exception caught → execution status set to <code>"failed"</code> with error message
Container times out	Timer fires → <code>docker rm -f</code> → <code>timedOut: true</code> in result
Output exceeds limit	Container force-stopped with "output limit exceeded" message
Container crashes	Exit code captured → status set to <code>"failed"</code>

5.12 Trade-offs Analysis

Aspect	Advantage	Disadvantage
Extensibility	New languages added with ~10 lines of config	Security scanner patterns are language-specific and need manual addition
Isolation	Perfect sandbox via Docker containers	Cold-start latency for first build of each language

Caching	Content-addressed images eliminate redundant builds	Docker image storage grows over time; no automated cleanup
Simplicity	Plain JS objects, no plugin framework needed	Profiles are static in code; no hot-reload without restart
Security	Non-root users, no network, read-only FS	Rule-based scanner is basic; sophisticated exploits could bypass it
Scalability	Each execution is independent	No execution queuing or concurrency limits

5.13 Why This Architecture Instead of Static Implementation

Alternative considered: Pre-installing all language runtimes on the backend server and running code directly.

Why rejected:

- **Security:** Running user code on the same server as the backend is extremely dangerous
- **Isolation:** No resource limits, no filesystem isolation, no network isolation
- **Scalability:** One bad execution (infinite loop, fork bomb) would kill the entire backend
- **Extensibility:** Adding new languages would require modifying the server's OS configuration
- **Reproducibility:** "Works on my machine" problems across different environments

The template-based Docker approach gives you **disposable, isolated, reproducible** execution environments with minimal configuration overhead.

6. Docker — Technical Deep Dive

6.1 Docker's Role in This Project

Docker serves **two completely different purposes** in iTECify:

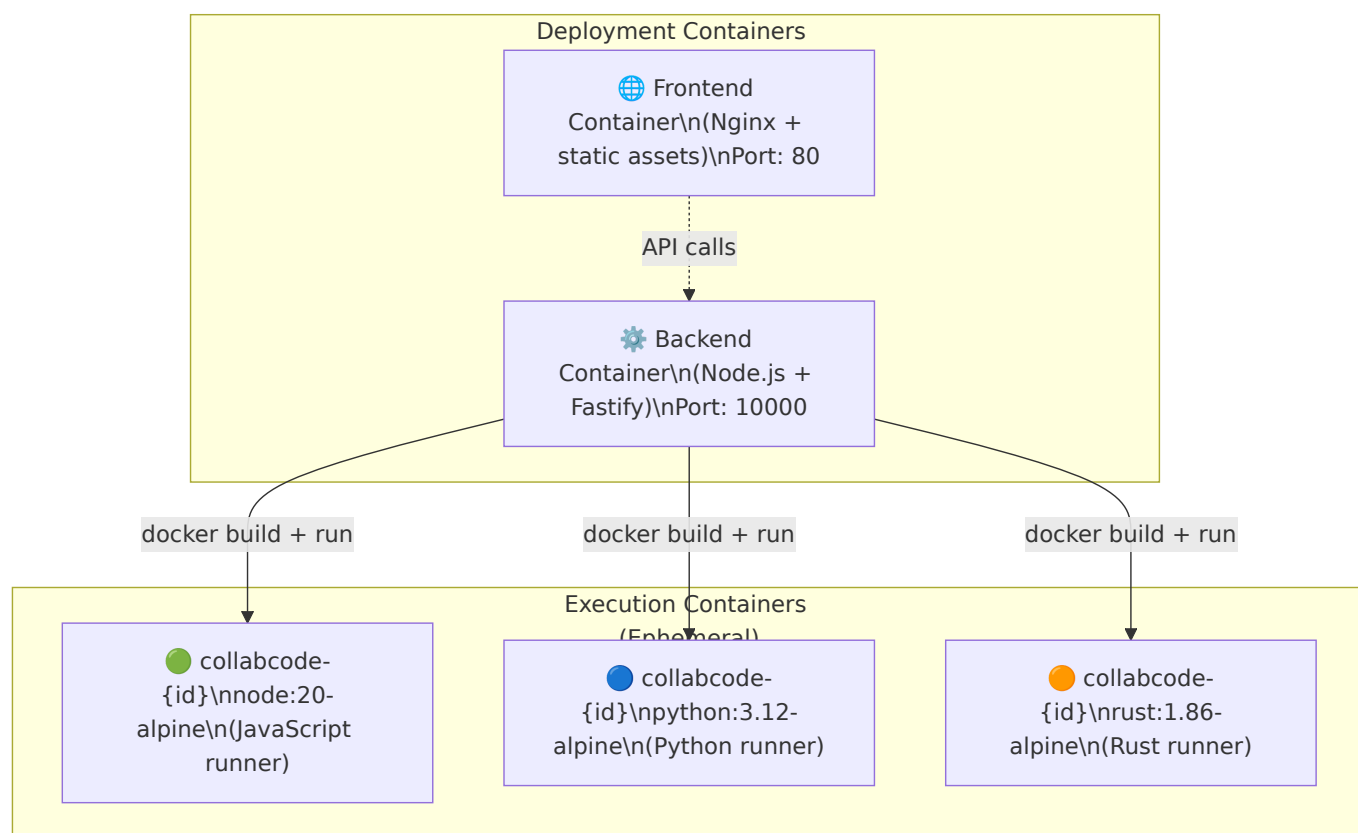
Purpose	What	Where
Application packaging	Containerize the frontend and backend for deployment	Root <code>Dockerfile</code> , <code>backend/Dockerfile</code>

Code execution sandboxing	Run user-submitted code in isolated, ephemeral containers	<code>dockerSandbox.js</code> , <code>languageProfiles.js</code>
----------------------------------	---	---

[!IMPORTANT]

These are fundamentally different use cases. The deployment containers are long-lived and serve the application. The execution containers are **ephemeral** — created per request, live for seconds, and are destroyed after use.

6.2 Container Inventory



6.3 Frontend Dockerfile — Multi-Stage Build

```
# Stage 1: Build (Node.js)
FROM node:18 as build
WORKDIR /app
COPY . .

# Accept Vite env vars as build args
ARG VITE_SUPABASE_URL
ARG VITE_SUPABASE_ANON_KEY
ARG VITE_GEMINI_API_KEY
ARG VITE_BACKEND_URL

# Set them as env vars for Vite (baked into bundle at build time)
ENV VITE_SUPABASE_URL=$VITE_SUPABASE_URL
ENV VITE_SUPABASE_ANON_KEY=$VITE_SUPABASE_ANON_KEY
ENV VITE_GEMINI_API_KEY=$VITE_GEMINI_API_KEY
ENV VITE_BACKEND_URL=$VITE_BACKEND_URL

RUN npm install
RUN npm run build

# Stage 2: Serve (Nginx)
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Key design decisions:

Decision	Reasoning
Multi-stage build	Build tools (Node, npm) are not included in the final image — only Nginx + static files
Build ARGs → ENV	Vite environment variables are inlined at build time (not runtime), so they must be passed as build args
Nginx alpine	Minimal image (~7MB) for serving static files

SPA fallback

`try_files $uri $uri/ /index.html` ensures React Router handles all routes

6.4 Backend Dockerfile

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 10000
ENV PORT=10000
CMD ["npm", "start"]
```

Key point: This container is simpler because the backend runs a Node.js server directly. No build step needed.

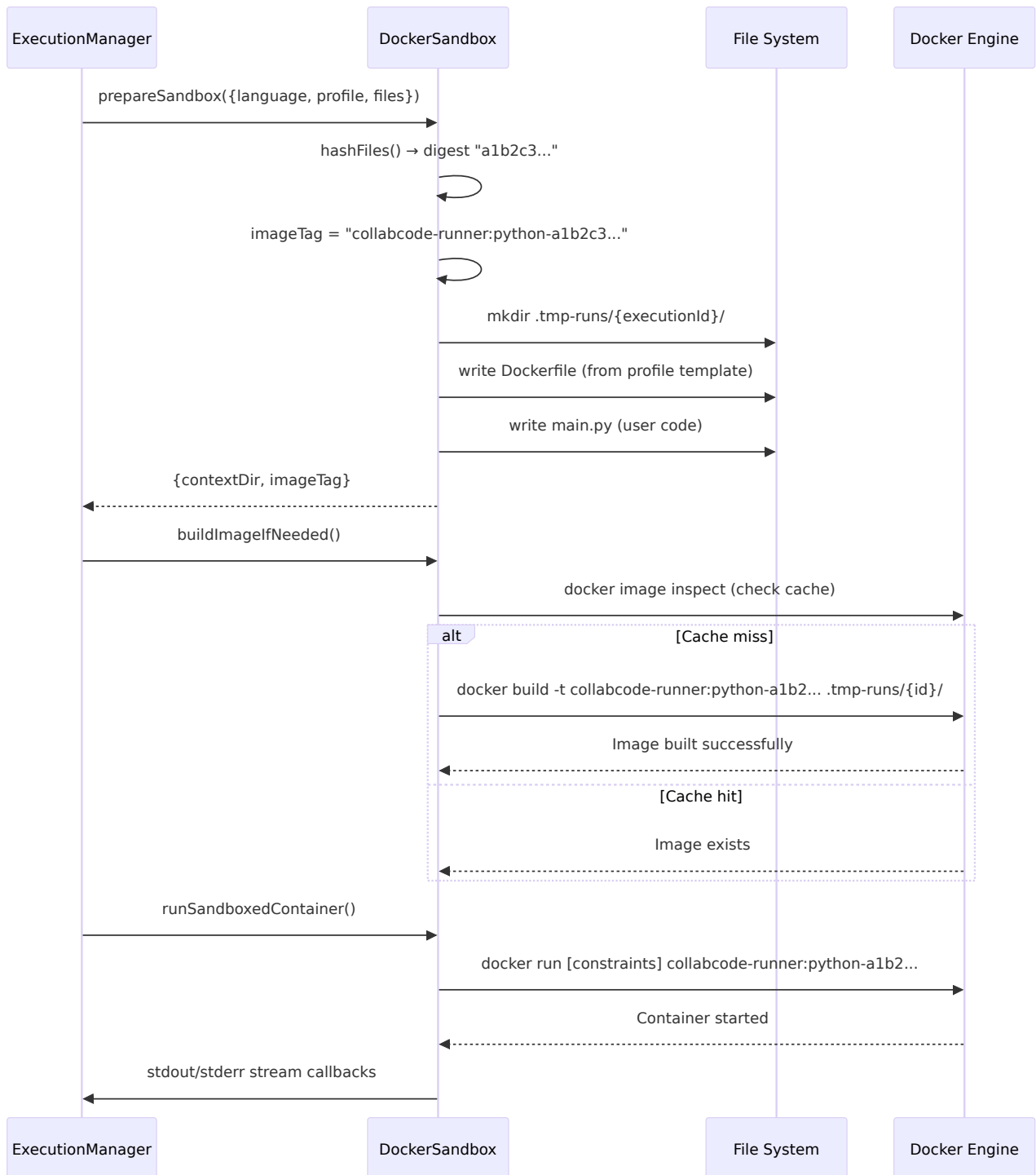
[!WARNING]

The backend container needs **Docker-in-Docker** or **Docker socket access** to spawn execution containers. The backend container must have the Docker CLI available and access to the host's Docker daemon (typically via `/var/run/docker.sock` mount).

6.5 Execution Containers — The Sandbox Architecture

This is where Docker plays its most critical and sophisticated role. Each code execution request creates a **fully isolated, ephemeral container**.

Container Creation Flow



Security Constraints Applied to Every Container

```

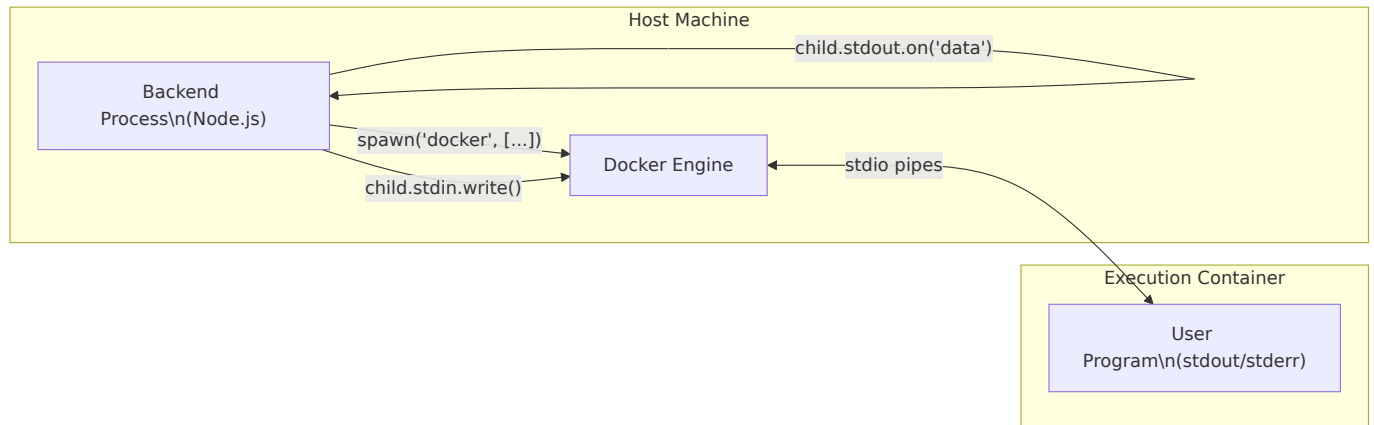
const args = [
  "run",
  "-i", // Interactive (stdin)
  "--name", containerName, // Unique name for management
  "--rm", // Auto-remove on exit
  "--network", "none", // ❌ No network access
  "--cpus", String(cpus), // CPU limit (default: 0.5
cores)
  "--memory", `${memoryMb}m`, // Memory limit (default:
256MB)
  "--pids-limit", "128", // Process limit (prevent fork
bombs)
  "--read-only", // ❌ Read-only root filesystem
  "--tmpfs", "/tmp:rw,noexec,nosuid,size=64m", // Writable /tmp, no
exec
  "--security-opt", "no-new-privileges", // ❌ No privilege escalation
  "--cap-drop", "ALL", // ❌ Drop ALL Linux
capabilities
  imageTag,
];

```

Constraint	What It Prevents
<code>--network none</code>	Calling external APIs, downloading malware, data exfiltration
<code>--read-only</code>	Writing to the container filesystem (except <code>/tmp</code>)
<code>--memory 256m</code>	Memory bombs, excessive allocation
<code>--cpus 0.5</code>	CPU-intensive operations monopolizing the host
<code>--pids-limit 128</code>	Fork bombs
<code>--cap-drop ALL</code>	Privilege escalation, kernel exploits
<code>--no-new-privileges</code>	setuid/setgid escalation
<code>--tmpfs /tmp:noexec</code>	Writing executable files to temp directory

<code>--rm</code>	Orphaned containers consuming resources
Timeout (10s)	Infinite loops, hanging processes
Output limit (200KB)	Infinite output loops

6.6 Container Communication



The backend communicates with execution containers through:

1. **Docker CLI** (`child_process.spawn`) — not the Docker API; simpler, no SDK dependency
2. **stdio pipes** — stdout/stderr are piped back to the Node.js process
3. **stdin pipe** — user input can be sent to the running container

There are **no Docker networks** between containers. Each execution container is completely isolated (`--network none`).

6.7 Volumes and Storage

Volume/Path	Purpose	Container Type
<code>.tmp-runs/</code>	Temporary build context (Dockerfile + user files)	Host filesystem, used by backend
<code>/usr/share/nginx/html</code>	Built frontend assets	Frontend container
No persistent volumes	Execution containers have no persistent storage	Execution containers

The `.tmp-runs/` directory is cleaned up after each execution via `removeSandboxContext()`.

6.8 Environment Variables and Configuration

Frontend container (build-time):

Variable	Purpose
VITE_SUPABASE_URL	Supabase project URL
VITE_SUPABASE_ANON_KEY	Supabase anonymous key
VITE_GEMINI_API_KEY	Google Gemini API key
VITE_BACKEND_URL	Backend API URL

Backend container (runtime):

Variable	Default	Purpose
PORT	8787	Server listen port
HOST	0.0.0.0	Server bind address
ALLOWED_ORIGIN	http://localhost:5173	CORS allowed origin
EXECUTION_TIMEOUT_MS	10000	Max execution time
EXECUTION_MEMORY_MB	256	Container memory limit
EXECUTION_CPUS	0.5	Container CPU limit
MAX_CODE_SIZE_BYTES	200000	Max code payload size
MAX_FILES	20	Max files per execution
MAX_OUTPUT_BYTES	200000	Max stdout/stderr output

6.9 Docker Compose — Not Used

[!NOTE]

The project does **not** use Docker Compose. The frontend and backend are deployed **independently** — the frontend as a static site (via Nginx container or Vercel), the backend as a standalone Node.js container (with Docker socket access for spawning execution containers).

This is a deliberate choice:

- The frontend is a static SPA that can be deployed anywhere (CDN, Vercel, Nginx)
- The backend requires Docker socket access, which complicates Compose setups
- Supabase is a fully managed service — not self-hosted

6.10 What Docker Solves in This Project

Problem	How Docker Solves It
Running untrusted user code	Complete isolation via containers with strict constraints
Multi-language support	Each language gets its own container image with the correct runtime
Security	Network isolation, capability dropping, resource limits, read-only FS
Reproducibility	Same Dockerfile → same environment everywhere
Cleanup	<code>--rm</code> flag ensures containers are destroyed after use
Resource control	Memory, CPU, PID, and output limits prevent abuse
Deployment consistency	Frontend and backend packaged identically across environments

6.11 What Would Be Harder Without Docker

Without Docker	Consequence
Running user code directly on the host	Security catastrophe — full system access

Installing all runtimes on the backend server	Maintenance burden, version conflicts, OS-level changes
Resource limiting with OS tools	Complex cgroups/namespace setup instead of simple Docker flags
Multi-language support	Would need <code>nvm</code> , <code>pyenv</code> , <code>rustup</code> all managed on one server
Cleanup after execution	Manual process cleanup, temp file management
Deployment	"Works on my machine" problems across dev/staging/prod

6.12 Container Lifecycle Summary

```

Host
├── Docker Engine
│   ├── Frontend Container (nginx:alpine)
│   │   ├── Static HTML/JS/CSS
│   │   └── Port 80 → Browser
│   ├── Backend Container (node:18)
│   │   ├── Fastify API server
│   │   ├── Port 10000
│   │   └── Has Docker CLI access
│   └── Execution Containers (ephemeral)
│       ├── collabcode-{uuid1} (node:20-alpine) ← Lives 0-10
│       │   seconds
│       ├── collabcode-{uuid2} (python:3.12-alpine) ← Lives 0-10
│       │   seconds
│       └── collabcode-{uuid3} (rust:1.86-alpine) ← Lives 0-10
│           seconds

```

7. Key Architectural Decisions

Decision 1: Supabase as Backend-as-a-Service

Aspect	Detail
What	Use Supabase instead of a custom backend for auth, database, and real-time
Alternative	Custom Express/Fastify server with PostgreSQL, JWT auth, Socket.IO
Why chosen	Eliminates ~80% of typical backend code; built-in RLS, auth, realtime
Benefit	Faster development; managed infrastructure; automatic scaling
Trade-off	Vendor dependency; Supabase channel limits; less control over real-time protocol

Decision 2: Docker CLI Instead of Docker SDK

Aspect	Detail
What	Use <code>child_process.spawn("docker", [...])</code> instead of <code>dockerode</code> or Docker REST API
Alternative	dockerode (Node.js Docker SDK)
Why chosen	Zero additional dependencies; simpler debugging (<code>docker</code> commands are inspectable); CLI is always available
Benefit	No SDK version compatibility issues; easy to test manually
Trade-off	String-based API; need to parse CLI output; slightly more overhead per call

Decision 3: Content-Addressed Image Caching

Aspect	Detail
What	Tag Docker images with SHA256 hash of (language + file contents)
Alternative	Always rebuild; or use execution ID as tag
Why chosen	Same code = same image = no rebuild needed
Benefit	Dramatic performance improvement for repeated runs of the same code

Trade-off	Docker image storage grows; no built-in cleanup of stale images
------------------	---

Decision 4: Separate Frontend and Backend Deployment

Aspect	Detail
What	Frontend deployed as static SPA (Vercel/Nginx), backend as separate API server
Alternative	Monolithic server serving both API and static files
Why chosen	Decoupled scaling; frontend can use CDN; backend needs Docker access
Benefit	Independent deployments; frontend can be on Vercel for free; backend on Docker-capable host
Trade-off	CORS configuration needed; two deployment targets to manage

Decision 5: Zustand Over Redux

Aspect	Detail
What	Use Zustand for state management instead of Redux/Redux Toolkit
Alternative	Redux + RTK, MobX, Jotai
Why chosen	Minimal boilerplate; no providers/context needed; direct store access
Benefit	~90% less code than equivalent Redux setup; simpler mental model
Trade-off	Less middleware ecosystem; no Redux DevTools (though Zustand has its own)

Decision 6: Pre-Execution Security Scanning

Aspect	Detail
What	Rule-based regex scanner that runs before every code execution
Alternative	No scanning (rely solely on Docker isolation); AST-based analysis

Why chosen	Defense in depth — catch obvious threats before wasting Docker resources
Benefit	Blocks <code>child_process.exec</code> , <code>os.system</code> , subprocess calls before they reach Docker
Trade-off	Regex patterns are bypassed with obfuscation; may produce false positives

Decision 7: Supabase Realtime for Collaboration (Not Custom WebSocket)

Aspect	Detail
What	Use Supabase Realtime Channels for cursor/code sync instead of custom WS server
Alternative	Socket.IO, Yjs + custom signaling server
Why chosen	Already using Supabase; built-in presence tracking; no infrastructure to manage
Benefit	Zero additional server code for real-time collaboration
Trade-off	Limited to broadcast model (no OT/CRDT); potential conflicts on simultaneous edits

8. Complete End-to-End Scenario

Scenario: "Two developers collaboratively write and run a Python script"

Step 1: Alice creates a session

```
Alice → Browser → Landing Page → Login (Supabase Auth)
      → Dashboard → "New Session" (Python)
      → sessionStorage.createSession()
      → Supabase INSERT into sessions table
      → Redirect to /session/{sessionId}
```

Step 2: Session page loads

Session.jsx loads:

1. `sessionStore.joinSession(sessionId)` → Fetch session from Supabase
2. `collaborationStore.joinCollaboration()` → Subscribe to Supabase channel `"session:{id}"`
3. Monaco Editor initializes `session.code (JSON)` → Load file system from `session.code (JSON)`
4. Timeline events fetched `session_timeline_events` → Load history from `session_timeline_events`

Step 3: Bob joins via shared link

```
Bob → Browser → /session/{sessionId}
      → Auth check (ProtectedRoute)
      → Same load sequence as Alice
      → Supabase Presence: "join" event
      → Alice sees Bob's avatar in the collaborator bar
```

Step 4: Both write code simultaneously

Alice types on line 3:

- Monaco onChange
- collaborationStore.broadcastChanges(userId, path, changes)
- Supabase channel broadcasts "changes" event
- Bob's browser receives event
- Bob's Monaco applies changes (skipping Alice's userId)

Bob types on line 7:

- Same flow in reverse
- Alice sees Bob's changes appear

Cursor positions:

- collaborationStore.broadcastCursor()
- Both see colored cursors for the other user

Step 5: Alice asks AI for help

Alice opens AI Chat panel

- Types "How do I read a file in Python?"
- AiChat.jsx → POST to Gemini API (generativelanguage.googleapis.com)
- Response streamed back → Displayed in chat panel
- Alice copies code suggestion into editor

Step 6: Alice clicks "Run ►"

TerminalPanel.jsx:

1. Collect files from virtual file system
2. POST /v1/executions {language: "python", files: [{path: "main.py", content: "..."}]}
3. Receive {executionId: "abc-123", status: "queued"}
4. Open WebSocket: ws://backend/v1/executions/abc-123/stream

Step 7: Backend execution pipeline

```
ExecutionManager.createExecution():
```

1. `getLanguageProfile("python")` → {extension: "py", dockerfile: "FROM python:3.12-alpine..."}
2. `normalizeFiles()` → [{path: "main.py", content: "..."}]
3. Emit "queued" event

```
ExecutionManager.runExecution():
```

4. Status → "scanning"
5. `scanSourceBeforeRun()` → No high-severity findings → proceed
6. Status → "building"
7. `prepareSandbox()`:
 - `hashFiles()` → digest "f4e2a8..."
 - `imageTag` = "collabcode-runner:python-f4e2a8..."
 - Write `.tmp-runs/abc-123/Dockerfile`
 - Write `.tmp-runs/abc-123/main.py`
8. `buildImageIfNeeded()`:
 - `docker image inspect` → NOT FOUND
 - `docker build -t collabcode-runner:python-f4e2a8... .tmp-runs/abc-123/`
9. Status → "running"
10. `runSandboxedContainer()`:
 - `docker run -i --name collabcode-abc-123 --rm --network none \`
 `--cpus 0.5 --memory 256m --pids-limit 128 --read-only \`
 `--tmpfs /tmp:rw,noexec,nosuid,size=64m \`
 `--security-opt no-new-privileges --cap-drop ALL \`
 `collabcode-runner:python-f4e2a8...`

Step 8: Output streams back

```
Container stdout → child.stdout → onStdout callback
  → ExecutionManager.emitEvent({type: "stdout", chunk: "Hello,
Developer!"})
  → EventEmitter → WebSocket handler →
socket.send(JSON.stringify(event))
  → Browser WebSocket.onmessage → TerminalPanel renders output

Container exits (code 0):
  → Status → "completed"
  → emitEvent({type: "run-ended", exitCode: 0})
  → removeSandboxContext(.tmp-runs/abc-123/) → cleanup
```

Step 9: Bob sees the output too

```
Terminal state is broadcast via collaboration channel:
  → collaborationStore.broadcastTerminalSnapshot(userId, snapshot)
  → Bob's TerminalPanel receives and renders the terminal output
```

Step 10: Timeline event recorded

```
sessionStore.appendTimelineEvent({
  sessionId, actorUserId: alice.id,
  eventType: "execution",
  payload: {language: "python", status: "completed"}
})
→ INSERT into session_timeline_events
```

9. Final Presentation Structure

Slide Deck Blueprint

#	Section	Main Message	Visual	Key Verbal Points	Depth
---	---------	--------------	--------	-------------------	-------

1	Project Overview	"iTECify is a collaborative IDE in the browser"	Screenshot + one-sentence tagline	What, who, why	● Low
2	Problem / Purpose	"Collaborative coding shouldn't require setup"	Before/after comparison (screenshare vs. iTECify)	Pain points solved	● Low
3	High-Level Architecture	"Three independent tiers + managed BaaS"	Architecture diagram (Frontend → Backend → Docker ← Supabase)	Component overview	● Medium
4	Overall System Flow	"From login to running code — the full journey"	End-to-end flow diagram	Numbered sequence of events	● Medium
5	Main Components	"Each component has a clear responsibility"	Component table + interaction arrows	What each does and why	● Medium
6	Technologies	"The right tool for each job"	Technology grid (grouped by layer)	Language, framework, why chosen	● Low
7	Dynamic Modules	"Language profiles turn config into runtime environments"	Module architecture diagram + code snippet	How profiles work, lifecycle, caching, adding new languages	● High
8	Docker Deep Dive	"Docker is the security foundation of code execution"	Container diagram + security constraint table	Two uses of Docker, sandbox architecture, constraints, why Docker	● High

9	Key Decisions	"Every architectural choice has trade-offs"	Decision cards with pros/cons	Top 5 decisions + reasoning	 Medium
10	End-to-End Example	"Watch the system in action"	Sequence diagram (Alice + Bob scenario)	Walk through the entire flow	 Medium
11	Advantages & Trade-offs	"What we gain and what we accept"	Two-column comparison	Security, extensibility, performance, limitations	 Medium
12	Architecture Summary	"Simple config, powerful execution"	Final architecture diagram with all pieces connected	Revisit the big picture	 Low

Presentation Flow Principle

What is it? → Why does it exist? → How does it work? → What are the parts?

→ What technologies? → How do dynamic modules work? → How is Docker used?

→ Why these decisions? → See it in action → Summary

[!TIP]

Start with the **simplest explanation** a non-technical person could understand, then **progressively deepen** into the technical areas (dynamic modules and Docker) where the audience should appreciate the engineering decisions.

Appendix: Information Confidence Assessment

Area	Confidence	Notes
------	------------	-------

Frontend architecture	✓ High	All source files read and analyzed
Backend architecture	✓ High	All source files read and analyzed
Docker sandbox implementation	✓ High	Complete source code reviewed
Language profiles system	✓ High	Complete source code reviewed
Security scanner	✓ High	Complete source code reviewed
Database schema & RLS	✓ High	Full SQL setup script reviewed
Supabase Realtime collaboration	✓ High	collaborationStore fully reviewed
Deployment architecture	⚠ Medium	Dockerfiles present but no Docker Compose; Vercel config exists; actual deployment target unclear
Docker-in-Docker / socket access	⚠ Assumption	Backend Dockerfile doesn't mount Docker socket; this must be handled at deploy time
Production hosting	⚠ Unknown	No CI/CD configuration found; hosting platform not specified
Monitoring / logging	⚠ Unknown	Fastify logger enabled but no external monitoring integration found